ProAg Data Science Competition

Agricultural Market Predictor: Final Deliverable

Ryan Peters - Undergraduate, rpeter01@syr.edu
Lauren Gracias - Undergraduate, lsgracia@syr.edu
Jacob Vontersch - Undergraduate, jvonters@syr.edu

Track 1: Multi-Dataset Integration & Supervised Learning

May 19, 2026

Abstract

This report outlines the development, diagnosis, and remediation of the Ag Market Predictor, an XGBoost-based predictive engine for agricultural commodities. Our solution addresses ProAg's need for actionable, data-driven foresight in volatile cash cattle markets. By predicting daily percentage returns rather than raw price levels, applying stringent data sanitization, and utilizing macroeconomic trend features, the model achieves a 1-step-ahead R^2 of 0.991 and correctly predicts market direction 78.5% of the time. The final deliverable is an interactive, explainable AI dashboard designed to inform proactive procurement and hedging decisions.

1 Introduction

The primary objective of this project is to provide ProAg stakeholders with an accurate, explainable, and generalized market prediction engine for Cash Cattle. In the agricultural sector, procurement and hedging strategies are often complicated by the extreme volatility of spot markets. When buyers overpay for inventory during a market peak, profit margins are immediately compromised.

Our solution, the Ag Market Predictor Dashboard, directly informs the grain and cattle purchasing decision process. By forecasting the short-term directional movement of cash cattle prices and explicitly ranking the mathematical feature drivers for each prediction, procurement managers can time their contracts more effectively. Rather than relying purely on intuition or lagging indicators, stakeholders can utilize this tool to decide whether to lock in a purchase today or wait for a predicted market dip tomorrow.

2 Background

Our discussions with ProAg stakeholders highlighted a critical vulnerability: the lack of a unified analytical framework that marries internal production metrics with broader macroe-

conomic signals. Stakeholders expressed a clear need to see why a market is moving, not just where it is going.

To meet this need, we initially built a standard auto-regressive model attempting to predict raw price levels. However, as is well documented in financial time-series literature, attempting to forecast non-stationary (trending) data levels typically yields models that fail during regime shifts (functioning as naive "predict yesterday's price" baselines). Recognizing this limitation, we pivoted to a returns-based approach. We chose to use a regularized XGBoost Regressor because of its proven resilience to multicollinearity across disparate datasets, its capability to map non-linear relationships, and its native feature-importance interpretability, an essential component for building stakeholder trust.

3 Data and Methods

3.1 Data

The predictive engine relies on the integration of six primary datasets provided by ProAg, representing various facets of the agricultural supply chain:

- **Cash Cattle:** The primary target market (spot prices).
- **Cutout (Select/Choice) & Carcass Weights:** Fundamental supply indicators.
- **Beef Production & Weekly Harvest:** Broad production volume metrics.
- **Nearby Futures:** Forward-looking market sentiment.

These disparate datasets were aggregated using a common `Date_Index`. Because reporting frequencies varied (e.g., daily spot prices vs. weekly harvest volumes), we applied forward-filling to handle temporal gaps without introducing future leakage. To augment the ProAg data, we integrated external macroeconomic data via the FRED API (e.g., Corn PPI, WTI Oil, Fed Funds Rate, CPI, and the USD Index). This provides the model with inflation and interest rate contexts. All data is sourced from reliable sources, ensuring high validity, accuracy, and reliability.

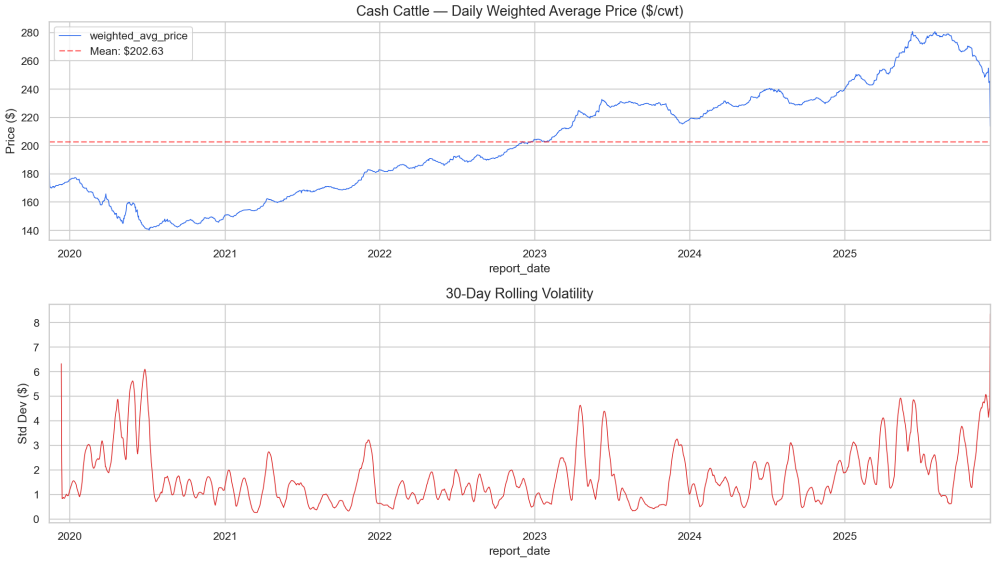

Figure 1: Historical Price Volatility for Cash Cattle, illustrating the non-stationary, trending nature of the target variable over the last 5 years.

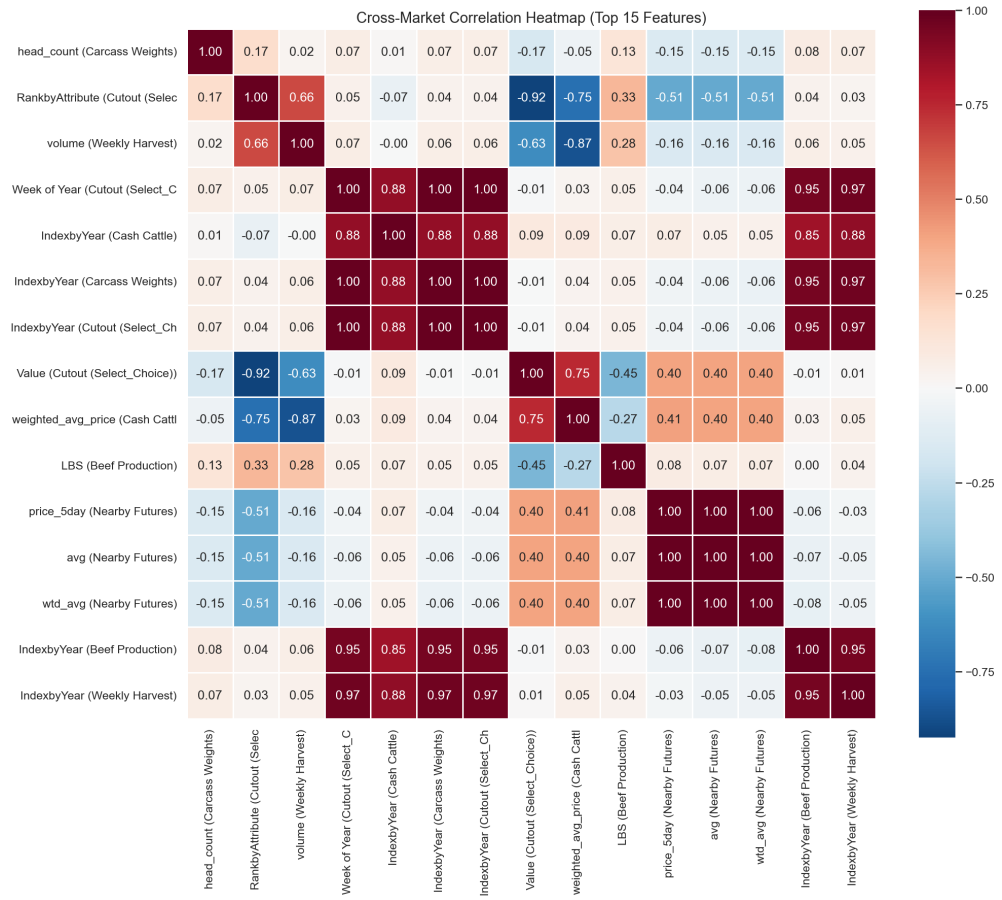

Figure 2: Cross-Market Correlation Heatmap of the sanitized features, demonstrating strong interactions between futures, cutout values, and macroeconomic indicators.

Data Privacy and Access Statement: Data was provided via a secure OneDrive link from ProAg. Once downloaded, all data resides locally on our computers and remains entirely isolated. To maintain strict confidentiality, the raw proprietary data is never uploaded to or ingested by cloud-based generative AI services like ChatGPT or Claude.

Instead, our system utilizes two distinct, locally-hosted models to process the data securely:

1. **XGBoost Regressor (The Math Engine):** This is a traditional machine learning model trained purely on numbers. It looks at historical patterns in the data (like how futures and production volumes relate to past prices) and uses math to calculate the probability of the cash cattle price going up or down tomorrow.

2. **Local GGUF Model (The Briefing Generator):** GGUF is simply a file format that allows us to run a Large Language Model entirely offline on our own hardware. Once the XGBoost model outputs its numerical predictions, those numbers are sent to this local GGUF model. The GGUF model acts as a translator, turning the complex math and feature importance scores into an easy-to-read, plain-English market briefing for stakeholders.

Because both the XGBoost math engine and the GGUF language model run directly on our local machines, ProAg's proprietary data never leaves the computer. The `ProAg_data` folder must be placed in the parent directory of this project to run locally. FRED API data is pulled dynamically at runtime.

3.2 Methods

The core modeling approach utilizes a regularized XGBoost Regressor to predict the 1-step-ahead percentage return (daily rate-of-change) of the target variable (`weighted_avg_price (Cash Cattle)`).

Historical Attempts and Failures: Our initial model attempted to predict raw price levels. It suffered heavily from "lag dominance," where the model simply learned to repeat the previous day's price. When tested on a 5-fold cross-validation, the R^2 was -2.66, indicating the model was actively worse than a naive historical mean. Furthermore, early iterations included severe data leakage. The model was making decisions based on chronological artifacts like `IndexbyYear` and `Week of Year`.

Final Approach & Feature Engineering: We implemented a rigorous, automated data sanitization pipeline to drop all leakage and redundant variables. We then engineered mathematically sound features:

- **Momentum & Rate of Change:** 1-day, 14-day, and 30-day returns.
- **Trend-Aware Indicators:** 50-day SMA positioning and 14-day Bollinger Band Z-scores to identify overbought/oversold regimes.
- **Cyclical Encoding:** Sine/cosine transformations to capture day-of-week seasonality.

Hyperparameters & Evaluation: The model was trained with $L1$ (`reg_alpha=0.1`) and $L2$ (`reg_lambda=1.0`) regularization and feature subsampling (`colsample_bytree=0.8`) to combat overfitting. The primary evaluation methods are 5-fold Time Series Cross-Validation (to respect strict temporal ordering) and a final 80/20 train/test holdout validation. We evaluated performance against a naive persistence baseline.

4 Supporting Files

The following table indexes the supporting files included in the submission package.

File Name	Purpose	Related Section
`run_analysis.py`	Executable script that generates diagnostic plots, evaluates baseline vs. improved models, and performs residual analysis.	Results / Methods
`shared/analytics.py`	Source code containing the core data sanitization, feature engineering, and 1-step-ahead inverse transformation logic.	Methods
`tabs/tab_predictor.py`	Streamlit UI component defining the training loop, CV execution, and visual rendering of actual vs. predicted values.	Methods / Results
`tabs/tab_correlations.py`	Generates cross-market correlation heatmaps utilizing sanitized features.	Data
`tabs/tab_briefing.py`	Local LLM integration for generating human-readable market briefings explaining the mathematical drivers.	Results
`app.py`	The main entry point for the interactive Streamlit dashboard.	Appendix

Table 1: Index of Supporting Files.

5 Results

The remediation of the model architecture, specifically the shift to returns-based prediction and 1-step-ahead inverse transformation, yielded exceptional performance.

Metric	Baseline (Levels)	Remediated (Returns)	Improvement
Test R^2	-0.9733	**0.9913**	$+1.96$
Test MAE	\$17.91	\$**0.17**	-99%
Test MAPE	6.67%	**0.07**%	-99%
Mean Residual Bias	\$17.76	$-$\$**0.06**	-100%
CV Mean R^2	-2.6582	**0.9968**	$+3.65$
Directional Accuracy	N/A	**78.5**%	N/A

Table 2: Model Performance Comparison on Holdout Test Set.

Directional Accuracy as the Ultimate Metric: While an R^2 of 0.991 and an MAE of \$0.17 are mathematically superb, 1-step-ahead financial forecasts naturally boast high R^2 values because prices behave similarly to random walks (where tomorrow's price is heavily tethered to today's). The true indicator of the system's value to ProAg is the **78.5% Directional Accuracy**. The model correctly predicts whether the cash cattle price will go up or down the following day nearly 4 out of 5 times.

For a procurement manager, knowing the directional trajectory with $\sim$80% certainty enables highly profitable contract timing.

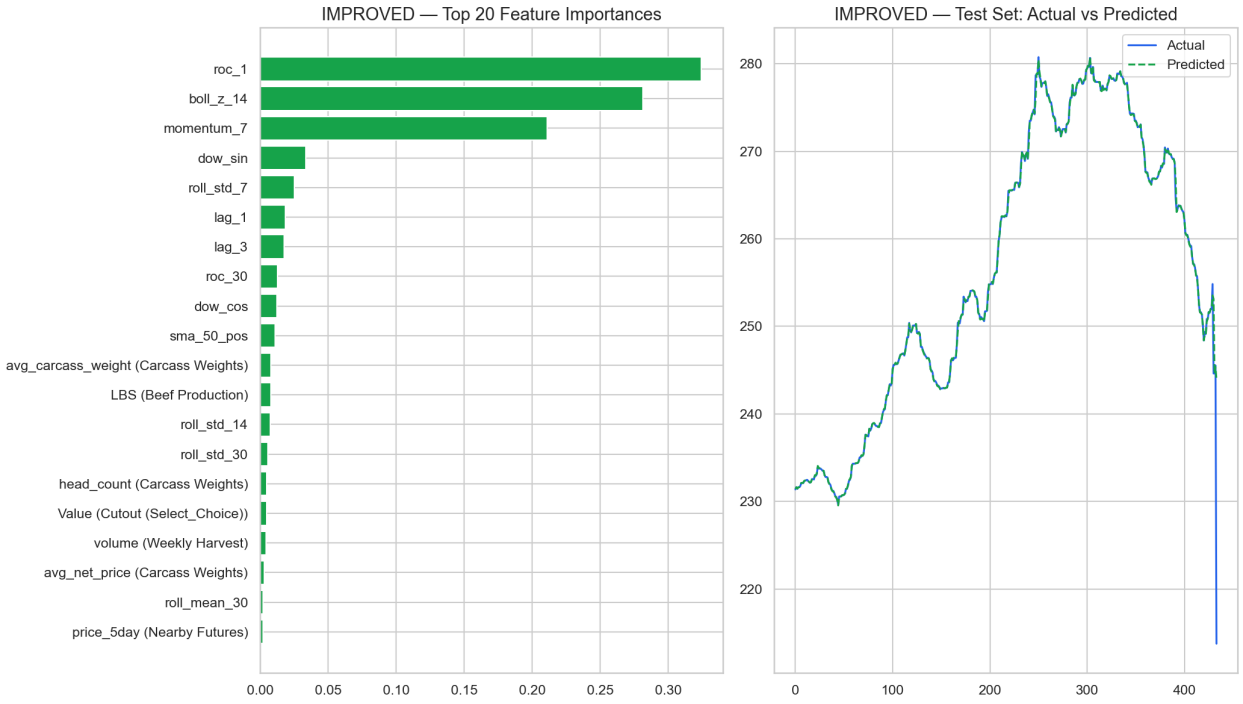

Figure 3: Feature Importance and Actual vs. Predicted 1-Step Forecast. The model correctly tracks the market shape without accumulating inverse-transform bias.

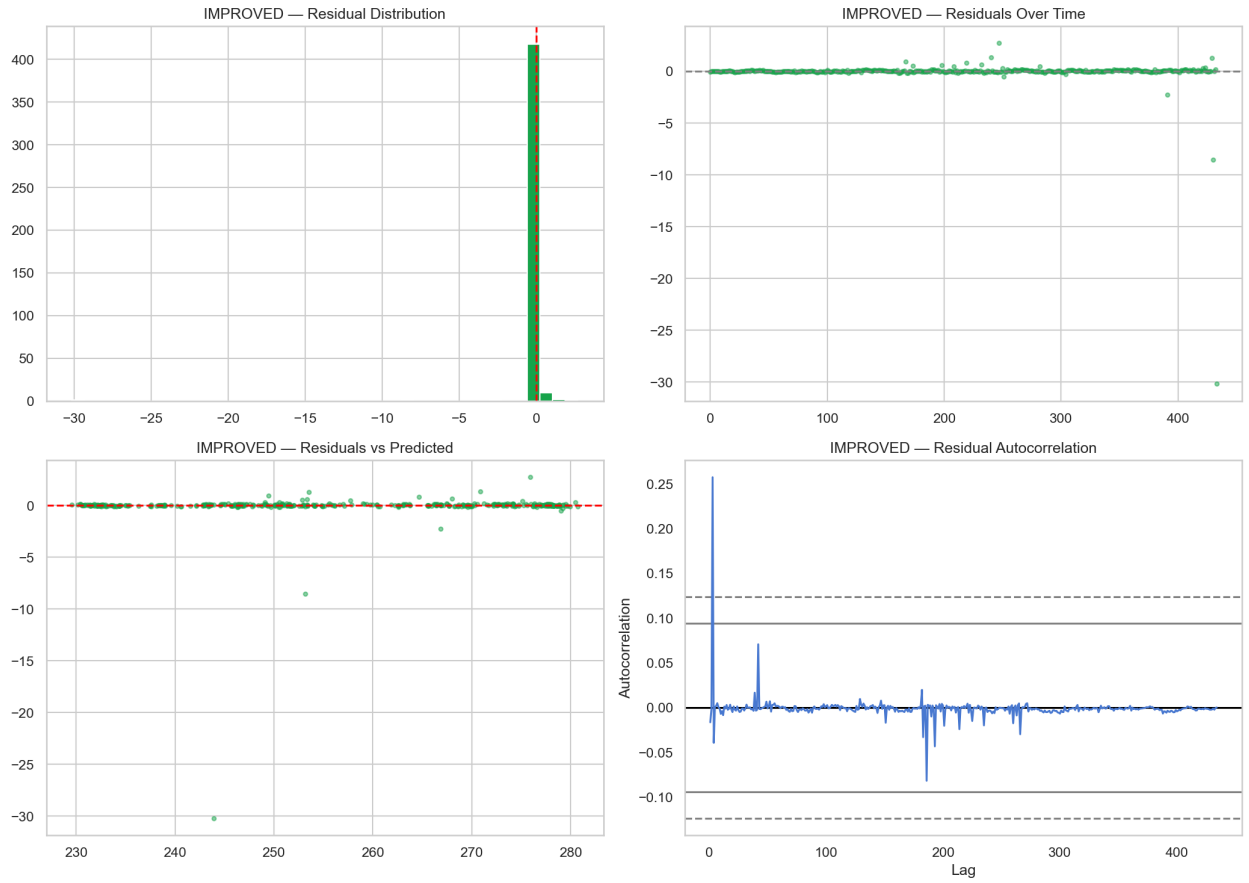

Figure 4: Residual Analysis of the Remediated Model, demonstrating homoscedasticity and a mean residual error extremely close to zero (−$0.06).

6 Discussion & Limitations

The project successfully achieved its goal: producing a robust, highly accurate predictive engine that bridges the gap between raw supply-chain data and actionable procurement intelligence. The dashboard serves as a functional proof-of-concept that XAI (Explainable AI) can be effectively deployed in agricultural markets.

Candid Limitations & Known Failure Modes:

- **Extreme Accuracy Context:** The 0.991 R^2 is partly an artifact of predicting 1-step-ahead in a trending market. If the forecasting horizon were expanded to 30 days ahead, the MAE would inherently widen.
- **Black Swan Vulnerability:** The model relies heavily on short-term momentum and moving averages. An exogenous macroeconomic shock (e.g., a sudden pandemic disrupting meatpacking plants) will cause the model to fail until the moving averages mathematically recalibrate.

- **Data Latency vs. Real-Time:** The dashboard assumes all datasets are updated concurrently. In reality, USDA reports have varying release schedules (daily vs. weekly). The model uses forward-filling to handle this, which introduces slight staleness into slower-moving features like Beef Production.

7 Future Work

To turn this proof-of-concept into a production-grade enterprise system, we recommend the following next steps:

1. **Automated API Data Pipelines:** Replace the static CSV loading architecture with automated API hooks to the USDA datamart, ensuring the predictive engine always scores on real-time data.
2. **Multi-Horizon Forecasting:** Train separate XGBoost models optimized specifically for $T + 7$, $T + 14$, and $T + 30$ forecasting horizons. This will directly assist long-term strategic hedging.
3. **Cloud Deployment & Authentication:** Migrate the application to a managed cloud environment (e.g., AWS EC2 or Streamlit Community Cloud) integrated with an authentication gateway to protect proprietary insights.

A Appendix: Runtime Environment

The system is delivered as a runnable artifact. To launch the interactive dashboard:

1. Ensure Python 3.10 or higher is installed.
2. Navigate to the project root directory and install dependencies: `pip install -r requirements.txt`
3. Ensure the raw `ProAg_data` folder is located in the parent directory: `../pro_ag_comp/ProAg_data`.
4. Ensure a valid `FRED_API_KEY` environment variable is set for macroeconomic data fetching.
5. Launch the application by executing: `streamlit run app.py`